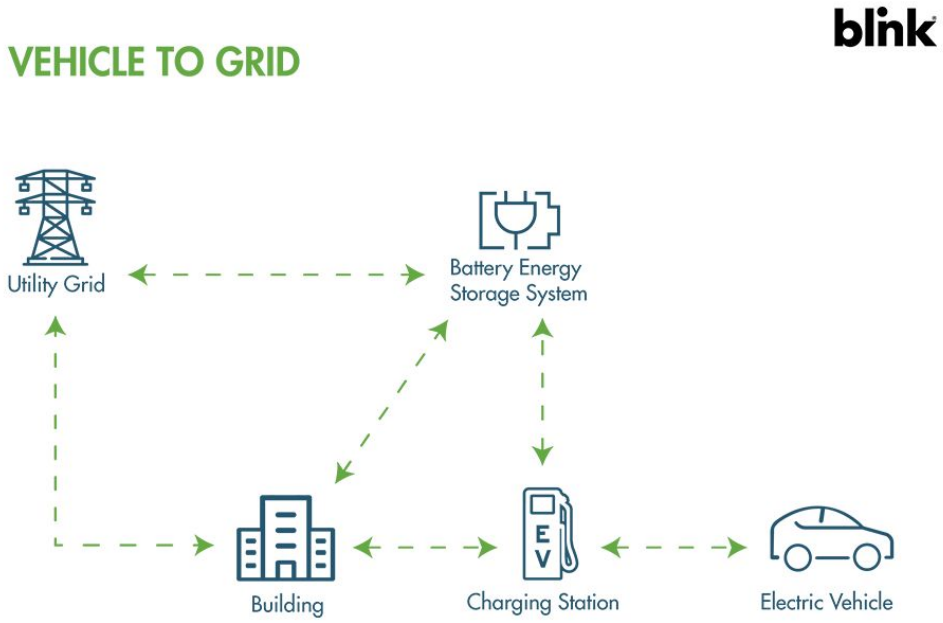
V2G Scheduler API

App prototype under project: Economic Feasibility of Vehicle to Grid V2G in Hungary

Team
Shayan Ghiaseddin
Sofia Nikolaeva
Nurzada Nasirova
MSc Business Informatics | Corvinus University of Budapest

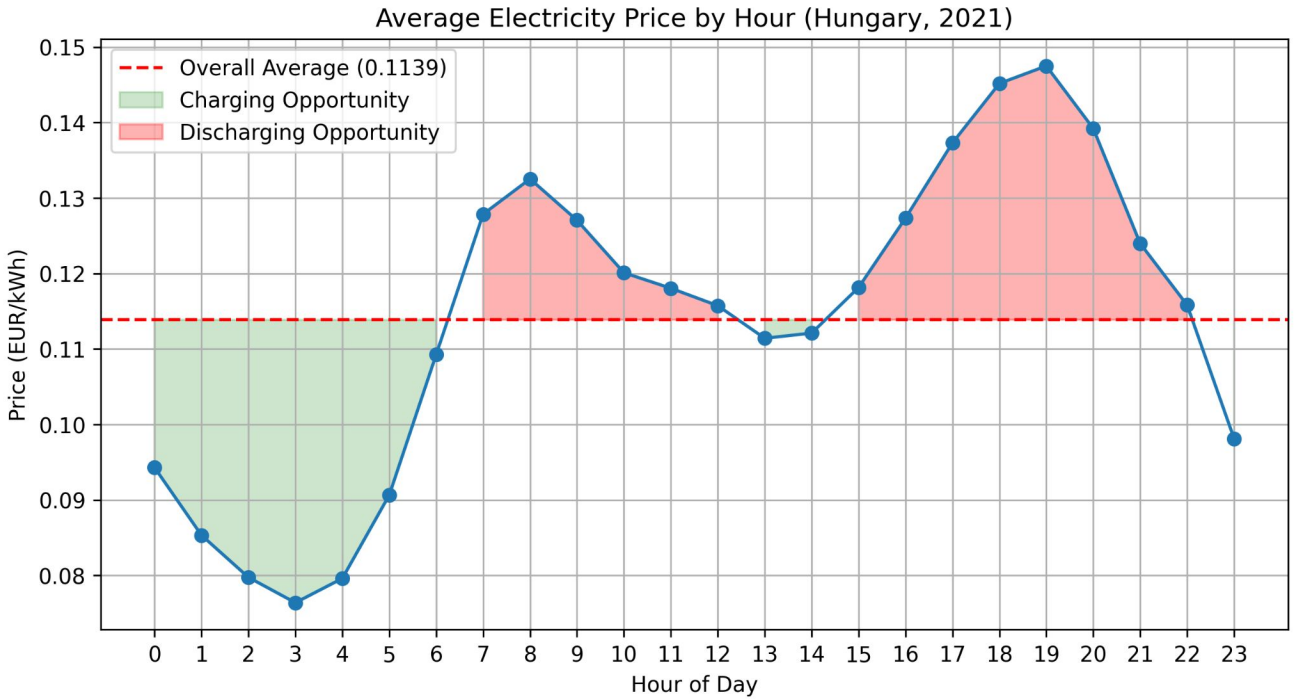
Course
Data Science Project in Business, by Associate Professor Tibor Kovács | Spring 2026

V2G: Vehicle to Grid

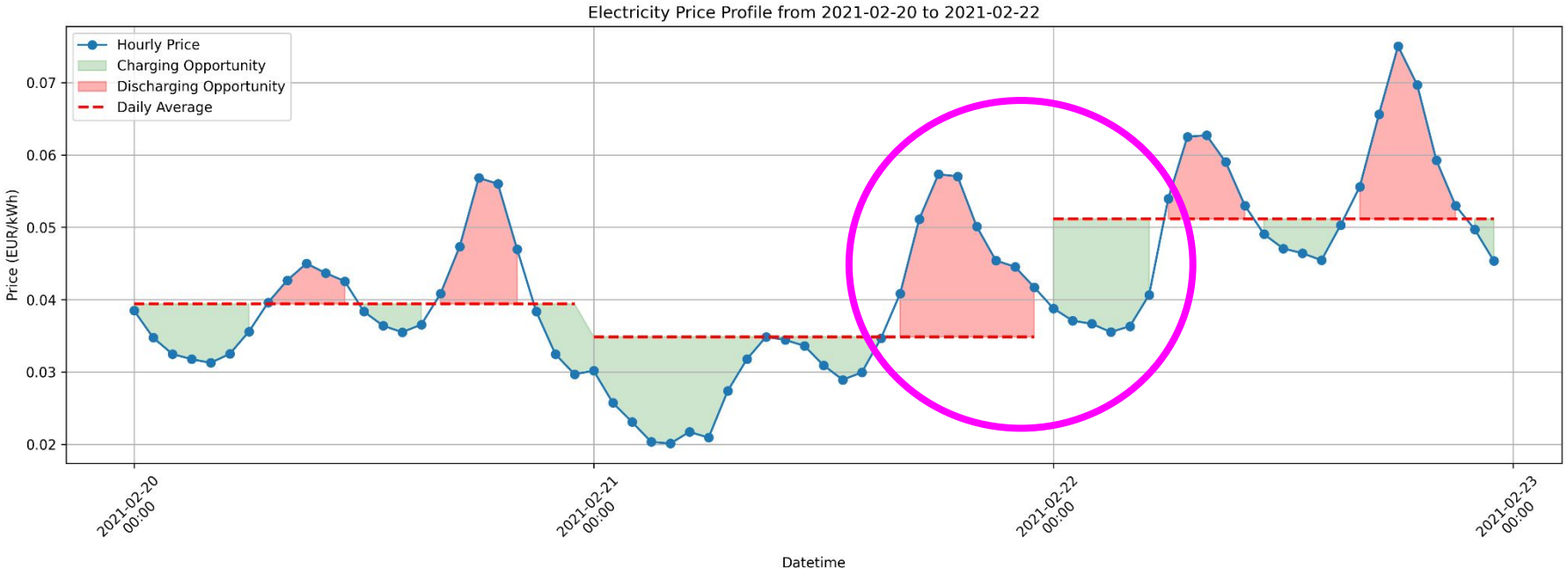


https://blinkcharging.com/blog/what-is-v2g

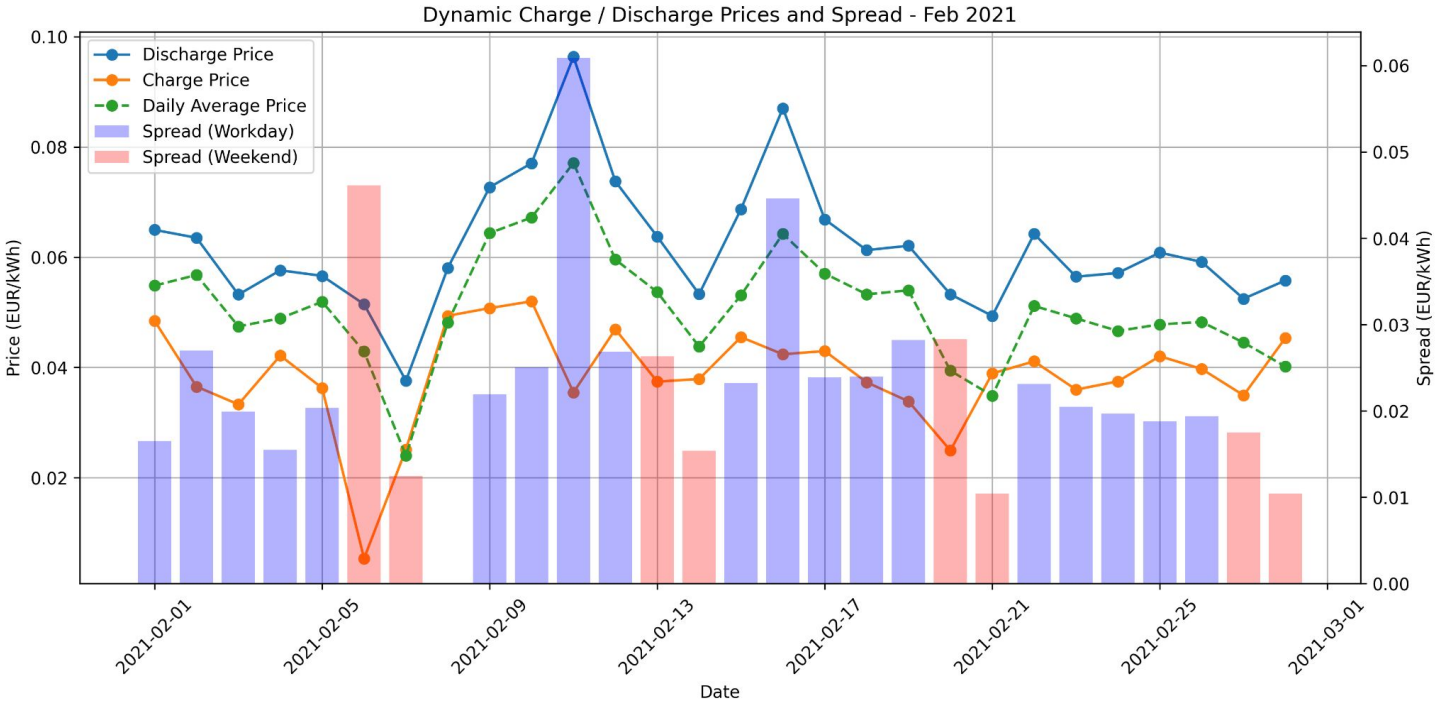
Can you see the arbitrage?



Sometimes “do nothing” is the best decision!



Daily Arbitrage Spread



Feasibility Study

Year	Annual_Profit_EUR	Annual_Profit_per_EV_EUR
2021	846,301	42
2022	20,837,415	692
2023	2,708,719	67
2024	24,610,655	413
2025	12,436,092	151
2026	14,266,896	135
2027	18,284,628	142
2028	22,464,992	151
2029	26,537,051	161
2030	30,406,231	172

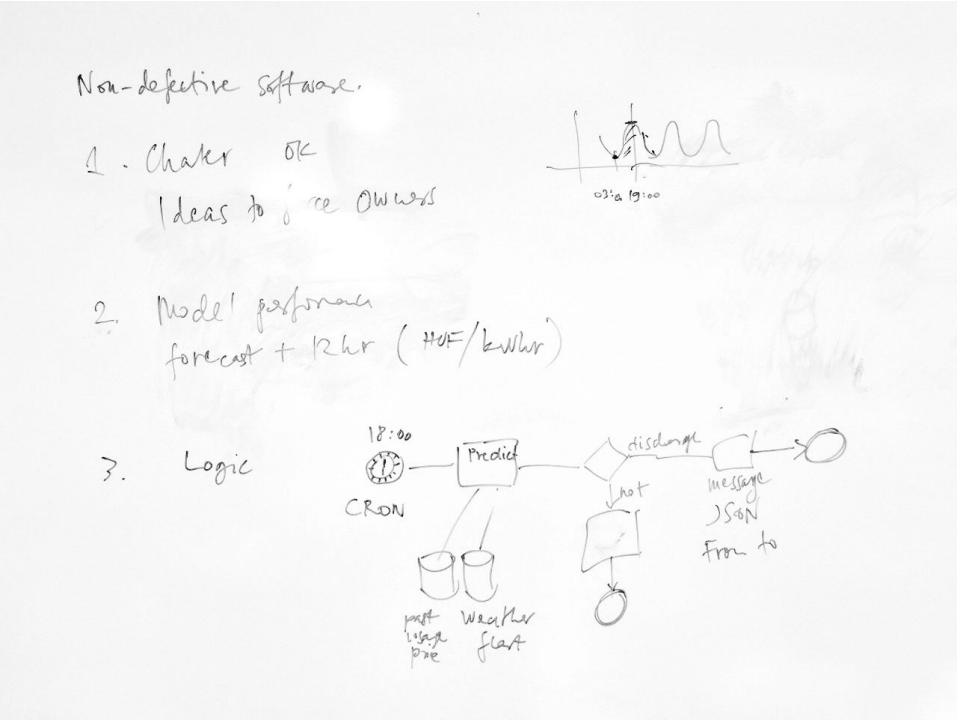
Metric	Value
Total income, 2021-2030 (EUR)	525,571,249
Total cost, 2021-2030 (EUR)	352,172,268
Total profit, 2021-2030 (EUR)	173,398,981
Average annual profit (EUR)	17,339,898
NPV, 2021-2030 @ 6% (EUR)	126,567,695

Blueprint of an App

A bidirectional station make a request to our app endpoint.

The response must be the schedule for the next 24 hours: discharge, charge, or do nothing.

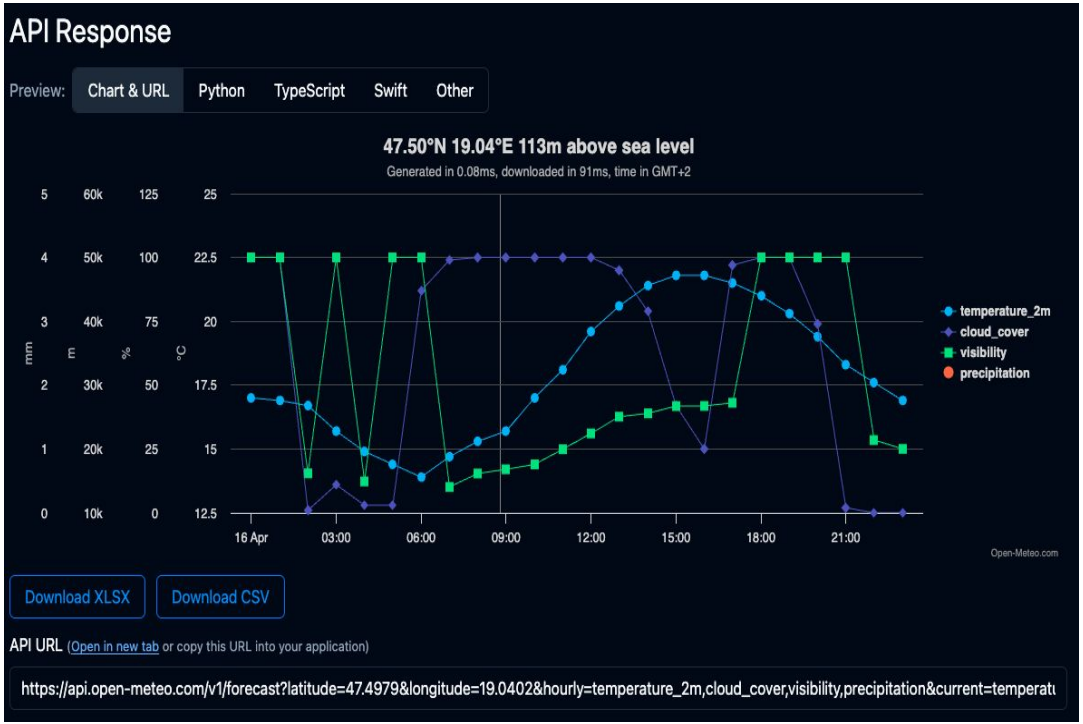
To make it more fancy, we adopt weather data into our training: https://www.kaggle.com/datasets/demetera/budapest-weather-history-from-2012



Historical Weather Data to Enrich Training

	Datetime	Price	temp	clouds	visibility	precipitation
0	2015-01-01 01:00:00	0.04416	-9.0	0.95	4.0	2.0
1	2015-01-01 02:00:00	0.03917	-8.4	0.95	4.0	2.0
2	2015-01-01 03:00:00	0.02693	-7.5	0.95	5.0	2.0
3	2015-01-01 04:00:00	0.02094	-7.0	1.00	5.0	2.0
4	2015-01-01 05:00:00	0.01852	-6.8	1.00	6.0	2.0

open-meteo.com



	Datetime	temp	clouds	visibility	precipitation	Price
0	2026-04-16 17:00:00	21.2	0.93	28.18	0.0	0.092098
1	2026-04-16 18:00:00	21.0	0.95	50.00	0.0	0.120594
2	2026-04-16 19:00:00	20.3	0.94	27.72	0.0	0.147626
3	2026-04-16 20:00:00	19.2	0.88	26.88	0.0	0.155733
4	2026-04-16 21:00:00	18.3	0.47	50.00	0.0	0.128619
5	2026-04-16 22:00:00	17.5	0.07	23.98	0.0	0.109359
6	2026-04-16 23:00:00	16.6	0.00	22.02	0.0	0.097616
7	2026-04-17 00:00:00	15.3	0.00	50.00	0.0	0.092796
8	2026-04-17 01:00:00	14.8	0.00	50.00	0.0	0.092033
9	2026-04-17 02:00:00	14.5	0.00	50.00	0.0	0.091484
10	2026-04-17 03:00:00	14.1	1.00	11.88	0.0	0.079902
11	2026-04-17 04:00:00	13.7	0.00	11.30	0.0	0.082685
12	2026-04-17 05:00:00	13.3	0.00	11.32	0.0	0.089276
13	2026-04-17 06:00:00	13.0	0.15	50.00	0.0	0.104369
14	2026-04-17 07:00:00	14.0	0.33	50.00	0.0	0.103706
15	2026-04-17 08:00:00	15.1	0.00	50.00	0.0	0.103914
16	2026-04-17 09:00:00	16.5	0.00	18.54	0.0	0.065666
17	2026-04-17 10:00:00	18.3	0.00	27.24	0.0	0.013895
18	2026-04-17 11:00:00	19.6	0.00	31.32	0.0	0.003393
19	2026-04-17 12:00:00	20.4	0.00	33.06	0.0	-0.000903
20	2026-04-17 13:00:00	21.7	0.00	35.34	0.0	-0.002650
21	2026-04-17 14:00:00	22.3	0.02	35.90	0.0	-0.003241
22	2026-04-17 15:00:00	22.5	0.03	35.58	0.0	0.006777
23	2026-04-17 16:00:00	22.5	0.54	33.94	0.0	0.049110

Schedule & Decision

-1 = discharge

0 = do nothing

1 = charge

jupyter app Last Checkpoint: 58 minutes ago

File Edit View Run Kernel Settings Help

Markdown

[17]:
```
request_ts = pd.Timestamp("2026-04-16 17:30:20")

command_schedule, prediction_df, snapshot_metrics = build_24h_command_schedule(
    request_timestamp=request_ts,
    historical_training_df_for_fallback=price_model_df,
)

command_schedule
```

[17]:
```
[{'2026-04-16 17:00:00': '0'},
 {'2026-04-16 18:00:00': '0'},
 {'2026-04-16 19:00:00': '-1'},
 {'2026-04-16 20:00:00': '-1'},
 {'2026-04-16 21:00:00': '-1'},
 {'2026-04-16 22:00:00': '0'},
 {'2026-04-16 23:00:00': '1'},
 {'2026-04-17 00:00:00': '1'},
 {'2026-04-17 01:00:00': '1'},
 {'2026-04-17 02:00:00': '1'},
 {'2026-04-17 03:00:00': '1'},
 {'2026-04-17 04:00:00': '1'},
 {'2026-04-17 05:00:00': '1'},
 {'2026-04-17 06:00:00': '1'},
 {'2026-04-17 07:00:00': '0'},
 {'2026-04-17 08:00:00': '0'},
 {'2026-04-17 09:00:00': '0'},
 {'2026-04-17 10:00:00': '0'},
 {'2026-04-17 11:00:00': '0'},
 {'2026-04-17 12:00:00': '0'},
 {'2026-04-17 13:00:00': '0'},
 {'2026-04-17 14:00:00': '0'},
 {'2026-04-17 15:00:00': '0'},
 {'2026-04-17 16:00:00': '0'}]
```

Create the App

```
Mac:app shayan$ uvico
INFO:     Will watch
-01/Data Science proj
app']
INFO:     Uvicorn run
INFO:     Started rel
INFO:     Started ser
INFO:     Waiting for
INFO:     Application
INFO:     127.0.0.1:5
```

Economic-Feasibility-of-Vehicle-to-Grid-V2G-in-Hungary

EXPLORER

ECONOMIC-FEASIBILITY-OF-VEHICLE-TO-GRID-V2G...

- .devops
- .env
- push-to-github.sh
- app
  - __pycache__
  - data
  - models
  - .dockerignore
  - app.py
  - config.py
  - dockerfile
  - README.md
  - requirements.txt
  - service.py
- data
- notebook
- output
- .gitignore
- LICENSE
- readme.md

app.py | service.py

app > app.py > predict

```python
14
15
16  class PredictRequest(BaseModel):
17      request_timestamp: Optional[datetime] = Field(
18          default=None,
19          description="If omitted, the API uses the current server time.",
20      )
21      lat: float = Field(default=BUDAPEST_LAT)
22      lon: float = Field(default=BUDAPEST_LON)
```

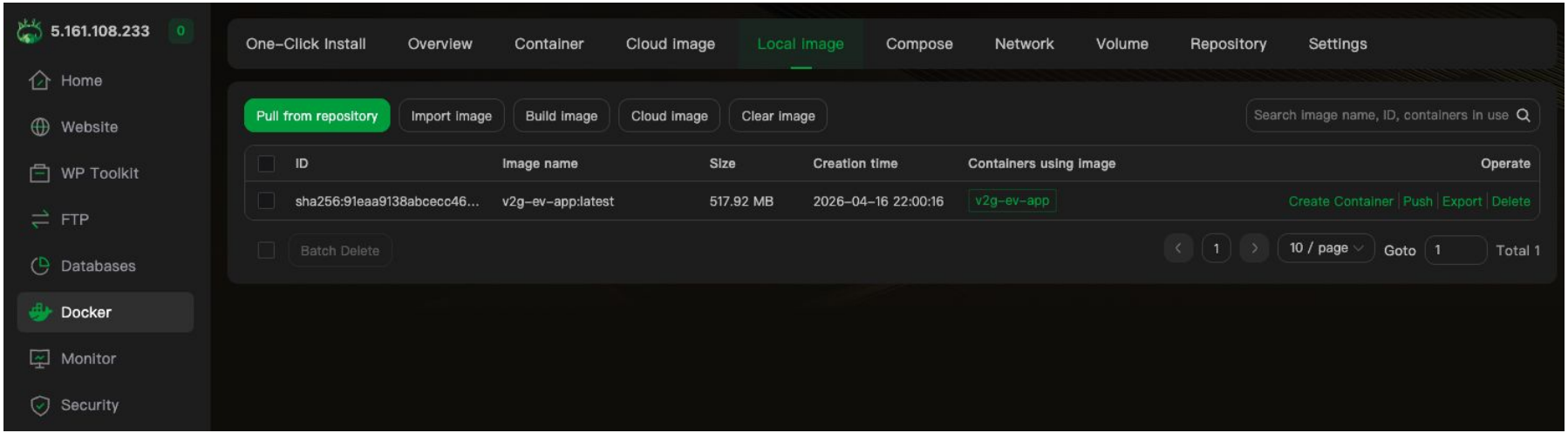
PROBLEMS OUTPUT TERMINAL PORTS AZURE

```
Mac:app shayan$ curl http://127.0.0.1:8000/predict
{"request_timestamp":"2026-04-16 23:36:41","snapshot_start":"2026-04-16 23:00:00","snapshot_end":"20
26-04-17 17:00:00","modeljoblib","pipeline_trained_at":"2026-04-16T18:54:07.220446Z","commands":[{"2
026-04-16 23:00:00":"0"},{"2026-04-17 00:00:00":"07 02:00:00":"0"},{"2026-04-17 03:00:00":"0"},{"202
6-04-17 04:00:00":"0"},{"2026-04-17 05:00:00":"0"},{"2026-04-17 06:00:00":"17 08:00:00":"0"},{"2026-
04-17 09:00:00":"0"},{"2026-04-17 10:00:00":"0"},{"2026-04-17 11:00:00":"0"},{"2026-04-17 12:00:00":
Mac:app shayan$
```

Dockerize & Deploy

Env: python:3.11-slim

Libraries: fastapi, uvicorn, pandas, numpy, scikit-learn, joblib, requests, pydantic



Vola!

Just click here and get your schedule:
https://sghiaseddin.com/project/v2g/predict

Repo: https://github.com/sghiaseddin/Economic-Feasibility-of-Vehicle-to-Grid-V2G-in-Hungary/tree/main/app

sghiaseddin.com/project/v2g/predict

Pretty-print


```
{
  "request_timestamp": "2026-04-16 22:13:03",
  "snapshot_start": "2026-04-16 22:00:00",
  "snapshot_end": "2026-04-17 17:00:00",
  "model_year": 2026,
  "pipeline_file": "v2g_pipeline.joblib",
  "pipeline_trained_at": "2026-04-16T20:05:29.688850Z",
  "commands": [
    {
      "2026-04-16 22:00:00": "-1"
    },
    {
      "2026-04-16 23:00:00": "1"
    },
    {
      "2026-04-17 00:00:00": "1"
    },
    {
      "2026-04-17 01:00:00": "1"
    },
    {
      "2026-04-17 02:00:00": "1"
    },
    {
      "2026-04-17 03:00:00": "1"
    },
    {
      "2026-04-17 04:00:00": "1"
    },
    {
      "2026-04-17 05:00:00": "1"
    },
    {
      "2026-04-17 06:00:00": "1"
    },
    {
      "2026-04-17 07:00:00": "0"
    },
    {
      "2026-04-17 08:00:00": "0"
    },
    {
      "2026-04-17 09:00:00": "0"
    },
    {
      "2026-04-17 10:00:00": "0"
    },
    {
      "2026-04-17 11:00:00": "0"
    },
    {
      "2026-04-17 12:00:00": "0"
    },
    {
      "2026-04-17 13:00:00": "0"
```